Kolejka

Kolejka jest to **struktura danych**, w której dostęp do danych jest możliwy tylko do pierwszego i ostaniego elementu. Kolejka jest określana mianem FIFO (First In First Out), czyli pierwszy wchodzący element jest pierwszym wychodzącym. Struktura queue jest używana wszędzie tam, gdzie przetwarzanie danych musi odbywać się w kolejności ich nadejścia.

Typowe zastosowania kolejki

- obsługa zadań w systemach operacyjnych,
- symulacje procesów,
- obsługa zdarzeń w programach,
- algorytmy grafowe (np. BFS – Breadth-First Search),
- zarządzanie zasobami.

Operacje na kolejce

Kolejka w STL'u posiada następujące operacje:

- **push** - umieszczenie nowego elementu na końcu kolejki;
- **pop** - usunięcie istniejącego elementu z początku kolejki;
- **empty** - informacja czy kolejka jest pusta;
- **size** - zwraca ilość elementów umieszczonych w kolejce;
- **front** - zwraca wartość pierwszego elementu w kolejce.
- **back** - zwraca wartość ostatniego elementu w kolejce.
- **swap** – zamienia zawartość dwóch kolejek (mogą być różnej wielkości, ale tego samego typu)

W przeciwieństwie do vector czy list, kolejka nie umożliwia dostępu do elementów w środku. Można pobrać tylko pierwszy i ostatni element. Jeśli trzeba przeglądać całą zawartość – należy użyć pętli z pop(), co jednocześnie opróżni kolejkę.

Jak wykorzystywać adapter kolejki w C++

Aby móc skorzystać z kolejki w C++, należy dołączyć plik nagłówkowy:

```
#include <queue>
```

Kolejnym krokiem, jaki musimy wykonać jest utworzenie kolejki. Zapis uogólniony wygląda tak:

```
queue < TYP_DANYCH > nazwa_kolejki;
```

Kolejka może przechowywać dowolny typ danych - mogą być to typy, które już znasz czyli:

```
queue < int > kolejka1;       //kolejka przechowująca liczby
queue < string > kolejka2;    //kolejka przechowująca stringi (czyli tekst)
```

Umieszczanie nowego elementu w kolejce (metoda: push)

Metoda **push** służy do umieszczenia nowego elementu na końcu kolejki.

```
1   #include<iostream>
2   #include <queue>
3   using namespace std;
4
5   int main()
6   {setlocale(LC_CTYPE, "Polish");
7       queue < int > kolejkaLiczb;
8       kolejkaLiczb.push( 123 );
9       kolejkaLiczb.push( 12 );
10      kolejkaLiczb.push( 55 );
11
12  return 0;}
```

Odczytywanie i modyfikacja pierwszego elementu kolejki (metoda: front)

Metoda **front** służy do odczytania lub modyfikacji wartości umieszczonej na początku kolejki.

```
1   #include<iostream>
2   #include <queue>
3   using namespace std;
4
5   int main()
6   {
7       queue < int > kolejkaLiczb;
8
9       cout << "Podaj liczbe: ";
10      int liczba;
11      cin >> liczba;
12
13      kolejkaLiczb.push( liczba );
14      kolejkaLiczb.push( 222 );
15      kolejkaLiczb.push( 555 );
16
17      cout << "Pierwsza liczba w kolejce to: " << kolejkaLiczb.front() << endl;
18      kolejkaLiczb.front() *= 2;
19
20      cout << "Zmodyfikowalem pierwsza liczbe w kolejce" << endl;
21      cout << "Pierwsza liczba w kolejce to: " << kolejkaLiczb.front() << endl;
22
23      kolejkaLiczb.front() = 1234;
24      cout << "Zmodyfikowalem pierwsza liczbe w kolejce" << endl;
25      cout << "Pierwsza liczba w kolejce to: " << kolejkaLiczb.front() << endl;
26
27      return 0;
28  }
```

Odczytywanie i modyfikacja ostatniego elementu kolejki (metoda: back)

Metoda **size** zwraca ilość elementów aktualnie znajdujących się w kolejce.

```
1  #include<iostream>
2  #include<queue>
3  using namespace std;
4
5  int main()
6  {
7      queue <int> kolejkaLiczb;
8      int liczba;
9      cin>>liczba;
10     kolejkaLiczb.push(liczba);
11     kolejkaLiczb.push(2);
12     kolejkaLiczb.push(3);
13
14     cout<<"Ostatnia liczba w kolejce to "<<kolejkaLiczb.back()<<endl;
15     kolejkaLiczb.back()*=2;
16     cout<<"Po modyfikacjioOstatnia liczba w kolejce to "<<kolejkaLiczb.back()<<endl;
17     kolejkaLiczb.back()=1234;
18     cout<<"Po modyfikacjioOstatnia liczba w kolejce to "<<kolejkaLiczb.back()<<endl;
19 return 0;
20 }
```

Sprawdzanie czy kolejka jest pusta (metoda: empty)

Metoda **empty** sprawdza, czy kolejka jest pusta.

```
1  #include<iostream>
2  #include <queue>
3  using namespace std;
4
5  int main()
6  {
7  queue < int > kolejkaLiczb;
8
9      if( kolejkaLiczb.empty() )
10     {
11         cout << "Kolejka jest pusta." << endl;
12     } else
13     {
14         cout << "W kolejce znajduje sie przynajmniej jeden element." << endl;
15     }
16
17     kolejkaLiczb.push( 1234 );
18     kolejkaLiczb.push( 124 );
19     kolejkaLiczb.push( 34 );
20     kolejkaLiczb.push( 4 );
21     if( kolejkaLiczb.empty() )
22     {
23         cout << "Kolejka jest pusta." << endl;
24     } else
25     {
26         cout << "W kolejce znajduje sie przynajmniej jeden element." << endl;
27     }
28     return 0;
29 }
```

Usunięcie istniejącego elementu z początku kolejki (metoda: pop)

Metoda **pop** służy do usunięcia istniejącego elementu z początku kolejki.

```
1   #include<iostream>
2   #include <queue>
3   using namespace std;
4
5   int main()
6   { queue < int > kolejkaLiczb;
7
8       int liczba = 0;
9       do
10      {
11          cout << "Podaj liczbe (0 - konczy wprowadzanie liczb): ";
12          liczba = 0;
13          cin >> liczba;
14          if( liczba != 0 ) kolejkaLiczb.push( liczba );
15
16      } while( liczba != 0 );
17
18      cout << "Liczby wyjete z kolejki: ";
19      while( kolejkaLiczb.empty() == false )
20      {
21          cout << kolejkaLiczb.front() << ", ";
22          kolejkaLiczb.pop();
23      }
24      return 0;
25  }
```

Ćwiczenie 1:

Napisz program, który:

1. tworzy kolejkę typu int,
2. dodaje do niej pięć liczb,
3. wyświetla pierwszy i ostatni element,
4. następnie usuwa kolejkę element po elemencie, wyświetlając każdy usunięty element.

Ćwiczenie 2:

Utwórz kolejkę string i pobierz od użytkownika kilka zdań (np. w pętli), a następnie:

1. wypisz wszystkie elementy kolejki,
2. przed każdym pop() sprawdzaj, czy kolejka jest pusta,
3. policz, ile elementów było początkowo w kolejce.

Ćwiczenie 3:

Zaimplementuj prosty system zgłoszeń (np. numerków klienta). Program ma:

1. przyjmować zgłoszenia (push()),
2. gdy użytkownik wpisze „obsłuż”, usunąć pierwsze zgłoszenie (pop()),
3. wyświetlać aktualnie obsługiwaną osobę (front()),
4. kończyć działanie po wpisaniu „koniec”.

Przykład działania:

```
Zgłoszenie: 101 dodane.
Zgłoszenie: 102 dodane.
Obsłuż > obsłużono zgłoszenie nr 101.
Aktualnie w kolejce: 1 osoba.
```

Ćwiczenie 4:

Napisz program, który:

1. pobiera od użytkownika kilka liczb i dodaje je do kolejki (queue<int>),
2. używa stosu (stack<int>), aby odwrócić kolejność elementów kolejki,
3. wyświetla wynik końcowy.